The two fixtures don't share an execution model, and neither belongs to the study they'd be run under.

Review of the Phase B0 boundary against the actual legacy scripts and strategy code. Question: can the proposed B1 facade reproduce both backtests without changing their research behavior?

BLOCKED

Fixture 1 places **zero** NT orders, so the proposed `trades.parquet` artifact is empty for it. Fixture 2's one-day command cannot represent a year-level policy run. And both would run under a sealed collector study whose contract they don't satisfy.

THE BLOCKING OBSERVATION

The two fixtures were chosen to prove generality, and they prove the opposite

```
$ # does each strategy place real NT orders?
strategies/score_fanning_strategy.py          submit_order / order_factory : 0
strategies/w4_exit_strategy.py                submit_order / order_factory : 0
backtests/baseline_flip_parity/strategy.py    submit_order / order_factory : 22   (W4's base class)
```

`ScoreFanningStrategy` is a **virtual evaluator**: it holds `self.evaluators: List[ThresholdEvaluator]` and dispatches 1s bars to them. It never submits an order. `engine.trader.generate_positions_report()` therefore returns an empty frame, and `bar_execution=True` — the sole justification given for modifying `engine_builder.py` — has no effect on it whatsoever. Its real output is per-policy virtual trade tables written from evaluator state:

```
run_staged_backtest.py:146-167
for evalr in strategy.evaluators:
    trades_list = evalr.trade_history + evalr.active_trades
    df = pd.DataFrame([{trade_id, entry_time, entry_px, direction, qty, atr,
                        exit_time, exit_px, exit_reason, pnl, is_open} ...])
    df.to_parquet(checkpoint_path / f"results_{evalr.name}.parquet")   # R5, R2.5
```

`W4ExitStrategy` inherits `BaselineFlipParityStrategy`, which does place orders, so it produces a genuine position stream *and* its own blended-PnL log. The two fixtures are on opposite sides of the execution boundary. A single `backtest/trades.parquet` + `metrics.json` contract cannot serve both, and B0 does not acknowledge the split.

ANSWERS

The five questions asked

QUESTION 1

Can these strategies legitimately run under `studies/Gemini_clean_maturity_flip_rolling_5m_productivity`?

NO — IT FORCES COLLECTOR SEMANTICS AND BREAKS THE SEAL

Strategy identity leaves the sealed set. The execution manifest resolves the strategy from the compiled contract — `spec.execution.strategy_class = strategies.flip_prediction_collector.FlipPredictionCollector` — not from `--strategy`. A `--strategy` override therefore executes code the seal does not cover, while `verify_preexec_audit_seal` still reports `VALID`. That directly reintroduces the class of defect closed in rounds 2–3.

Chronology is the collector's, not the strategy's. `train=[2021,2022,2023] · dev=[2024] · prohibited=[2025,2026]`. `run_staged_backtest.py` defaults to `2025-03-01 .. 2025-03-31` — prohibited under this contract. Worse, `BaselineFlipParityConfig.year` defaults to **2025**, and the proposed W4 command passes `policy`, `theta`, `N` but **not** `year`. The strategy would load 2025 predictions while replaying 2023 bars — a prohibited-year read that `data_plan` cannot see, because the strategy opens the parquet directly rather than through the catalog.

Other contract terms that don't apply:
`observation_policy.required_source_relation: equal`, the 60-name `feature_list` + `feature_list_sha256` enforced by `OutputManager.persist_collection`, and the `population` / `target` / `stratification` / `baseline` / `lineage` blocks. Under `--stage full` the collector's smoke gate would also demand `smoke_acceptance.json` — a collector artifact — before a W4 backtest could run.

The B1 file list invalidates the seal. `engine_builder.py` and `strategy_binding.py` are both inside the 53-file AST closure. B0's note that "no collector code will be modified" is true of the collector module and false in effect: any byte change to those two files changes the composite and invalidates the Gemini seal, both audit statuses, and the current smoke acceptance.

QUESTION 2

Are the one-day harness commands equivalent to the legacy fixtures?

NO FOR BOTH — AND STRUCTURALLY IMPOSSIBLE FOR W4

W4 cannot be represented by a one-day command. The legacy run is `--year 2023`: it loads `2022-12-27 .. 2023-12-31` and calls `engine.run(start=2023-01-01)` with *no end*. `year` is not a date filter — it is a strategy config field that selects the offline prediction universe (`weakness_checkpoint_predictions.parquet`, 222 MB, filtered `dt.dt.year == cfg.year`). A single day replays roughly 1/250th of the population, so every policy metric the W4 line exists to produce — win rate, profit factor, total PnL, B0–B5 comparison — is a different statistic. A one-day run is a smoke test of the plumbing, not a reproduction of the fixture.

Two config facts the proposed command drops. `entry_qty = 2 if policy == "B4" else 1` is derived in the runner, not the strategy; passing `--param policy=B4` alone silently trades 1 lot and breaks the scale-out policy. And 12 further `BaselineFlipParityConfig` fields (`sl_atr`, `tp_atr`, `ma_period`, `ma_type`, `trade_side`, `use_trailing_stop`, `be_trigger_atr`, …) are taken as class defaults by the legacy runner and must be pinned, not re-defaulted.

Undeclared side-effect write. `w4_exit_strategy.py:266` writes `backtests/results/w4_parity_{year}_{policy}.parquet` on stop, outside `runs/` and outside `OutputManager`'s control. A one-day harness run would silently overwrite the year-level artifact at that fixed path.

Fixture 1's command is closer but still not equivalent — the legacy invocation carries `--checkpoint-dir` (both an input and the output location) and a `--resume` path via `DailyStateCheckpointer.verify_manifest` plus `strategy.load_daily_checkpoint`. The proposed facade signature has no parameter for either.

Run-window semantics are unspecified and decisive. `collect.py` calls `engine.run()` with no arguments, replaying warmup bars as live. Both legacy scripts load a 5-day lead-in and then bound the run (`run(start=…)` for W4, `run(start=…, end=…)` for fanning). Whether NT dispatches pre-`start` data determines whether the lead-in warms indicators or is discarded — and the answer changes both fixtures' results. B0 must state the intended semantics and the baseline must record which one the legacy actually exhibits.

QUESTION 3

Minimum fixture baseline that must be captured before B1

SPECIFIED BELOW — NONE OF IT EXISTS YET

Capture by running each legacy command *unmodified* and freezing the result. Both fixtures need every row; the shapes differ only in the output section.

ITEM	FIXTURE 1 — SCOREFANNING	FIXTURE 2 — W4
Command	`run_staged_backtest.py --start-date 2023-03-03 --end-date 2023-03-03`, verbatim, plus `--checkpoint-dir` value	`run_w4_backtest.py --year 2023 --policy B1 --theta 0.62 --N 10`, verbatim
Data window	load `2023-02-26 .. 2023-03-03 23:59:59`; run window `start=2023-03-03, end=2023-03-03 23:59:59`	load `2022-12-27 .. 2023-12-31 23:59:59`; run `start=2023-01-01`, **no end**
Warmup	5 calendar days, applied as *load* extent only. Record bar counts loaded vs bar callbacks received per timeframe, so lead-in dispatch behaviour is pinned as an observation rather than an assumption.	
External inputs	`checkpoint_dir` contents + `DailyStateCheckpointer` manifest state (and whether `--resume` was inactive)	SHA-256 of `studies/regime_sequence_signal_audit/results/weakness_checkpoint_predictions.parquet`, plus the post-filter row count for `year==2023`
Catalog	`data/catalog/NQ_v0_2020_2026`; per-bar-type row counts and first/last `ts_event` and `ts_init` for the loaded window	
Instrument / venue	`NQ.XCME`, multiplier `20`, price increment `0.25`, activation `2019-01-01`, expiration `2027-12-31 23:59:59`; venue `XCME`, `OmsType.NETTING`, `AccountType.MARGIN`, base `USD`, starting balance `1,000,000 USD`, `bar_execution=True`, `bar_adaptive_high_low_ordering=True`. Record `trader_id` (`STAGED-BACKTESTER` / `W4-BACKTESTER`) and exclude it from comparison hashes.	
Strategy config	full resolved `ScoreFanningConfig` including the exact `policies` list (R5 @0.62, R2.5 @0.50, both `sl_atr_mult=1.5`, `pt_atr_mult=2.0`)	full resolved `W4ExitConfig` — all 19 inherited fields, with `year=2023` and `entry_qty` as derived, not as defaulted
Fills	**None** — assert the NT positions report is empty; that emptiness is part of the baseline	every fill: ts, side, qty, price, liquidity side, order type, commission
Closed positions	n/a — virtual trades only	`generate_positions_report()` filtered to `ts_closed.notna()`; row count and per-row fields
Open positions	count of evaluator trades with `is_open == True` at end of run	count of positions with `ts_closed` NA — recorded separately, since the legacy artifact *excludes* them
P&L / cost	per-policy: trade count, gross PnL, win rate, and the `exit_reason` distribution	total and per-trade PnL from both the positions report and `strategy.all_trades` (blended PnL for partial exits); commission and slippage totals
Outputs to hash	`results_R5.parquet`, `results_R2.5.parquet`	`trades.parquet`, `strategy_trades.parquet`, `w4_parity_2023_B1.parquet`

Tolerances

Timestamps, order/position counts, sides, quantities and `exit_reason` values must match **exactly** — a differing count is a behavioral change, not drift. Prices are tick-quantised at 0.25, so they should also match exactly; treat any deviation as a failure rather than a tolerance. Only aggregate float metrics (PnL sums, profit factor) may carry a tolerance, and `1e-9` relative is sufficient given identical inputs — a wider band would mask exactly the fill-model differences this exercise exists to detect. Hash the parquet *contents* (sorted, with volatile columns such as `trader_id` and run timestamps excluded), not the file bytes.

QUESTION 4

Should generic backtest execution reuse all of `data_plan.py`?

`NO — REUSE THE CATALOG/INSTRUMENT HALF ONLY`

`resolve_data_plan` bundles two unrelated responsibilities. The generic half is reusable: `PRODUCT_CATALOGS`, instrument and bar-type resolution, catalog path, the `ts_init` delta contract, and warmup arithmetic. Both legacy scripts hand-roll exactly this, and consolidating removes real duplication.

The governance half is not generic. Prohibited-year rejection, authorized-domain membership, the DEV/OOS unlock check and the warmup partition lock all derive from *the collector study's* `chronology` block. Applying them to an unrelated backtest imports one study's research partition into another's, and the score-fanning default window (2025-03) is already prohibited under it.

Recommended split, without new packages: extract the generic resolution into `resolve_catalog_plan(symbol, start, end, warmup_days)` inside the existing `data_plan.py`, and keep `resolve_data_plan` as the study-bound wrapper that calls it and then applies the chronology gates. Collect mode keeps calling the wrapper unchanged; backtest mode calls the inner function and applies whatever window its own contract declares. Enforcement stays attached to the contract that declares it.

QUESTION 5

Execution-mode contract instead of a `bar_execution` boolean

REQUIRED — AND IT MUST BE RECORDED, NOT JUST SET

A boolean records a switch position, not a semantic. This repository already has a recorded finding that bar-mode bracket simulation overstates fade strategies by $15–25K/yr — the fill model is a research-behavior parameter, so it belongs in the manifest as declared and hashed state.

Declare it in the study/run contract and echo it into `run_manifest.json`:

```
execution_mode:
  order_handling: virtual | simulated_orders     # fixture 1 vs fixture 2 — decides the output shape
  fill_model:     bar | tick
  bar_execution:  true
  bar_adaptive_high_low_ordering: true
  oms_type:       NETTING
  account_type:   MARGIN
  base_currency:  USD
  starting_balance: 1_000_000
  commission_model: <name or none>
  slippage_model:   <name or none>
  run_window:
    mode: bounded | all_loaded          # run(start,end) vs run() — differs from collect mode
    warmup_days: 5
    warmup_dispatched: true | false       # observed, not assumed
```

`order_handling` is the field that resolves the blocking finding: it selects which output contract applies, so a virtual-evaluator strategy is never asked to produce a position report and a real-order strategy can never silently emit an empty one.

REQUIRED B0 REVISIONS

Smallest set that unblocks B1 — document changes only, no new framework

R1 Declare two output contracts, keyed by `order_handling`

Replace the single `backtest/trades.parquet` + `metrics.json` block in §3 with two shapes: *virtual* (per-evaluator tables, one file per policy, carrying `is_open` and `exit_reason`) and *simulated_orders* (positions report filtered to closed, plus the strategy's own trade log, plus an explicit open-position count). State which fixture exercises which.

R2 Stop hosting these fixtures under the collector study

Remove `--study studies/Gemini_clean_maturity_flip_rolling_5m_productivity` from both fixture commands. Either give each fixture a minimal execution-only contract declaring its own instrument, window and strategy, or have backtest mode take those from the CLI. Also state explicitly that `--strategy` must never override a sealed study's declared `strategy_class` — if a study is sealed, the strategy it seals is the strategy that runs.

R3 Correct Fixture 2 to a year-level command and pin its config

The harness equivalent must be a 2023 full-year run with `--param year=2023` present. Document that `entry_qty` is policy-derived (`2` for B4, else `1`) and that the remaining 12 inherited config fields are pinned to their legacy-default values. Note the `w4_parity_{year}_{policy}.parquet` side-effect write as a known unmanaged output. If a one-day run is wanted as a plumbing smoke, label it as such — not as the fixture.

R4 Add a seal-invalidation note to §6

`engine_builder.py` and `strategy_binding.py` are inside the collector's 53-file sealed closure. State that modifying them invalidates the Gemini seal, both audit statuses and the smoke acceptance, and sequence the re-audit / re-seal / re-smoke as part of B1 rather than discovering it at execution time. The current note — "no collector code will be modified" — is misleading and should be corrected.

R5 Add the fixture baseline as a B0 deliverable

The table in Question 3 must be captured from unmodified legacy runs and frozen *before* any B1 code exists. Without it there is nothing to compare against, and "reproduces the legacy backtest" is unfalsifiable.

R6 Specify run-window semantics and the `data_plan` split

State whether backtest mode calls `engine.run(start, end)` or `engine.run()`, and record whether warmup bars are dispatched — this differs from collect mode and changes both fixtures. Add the `resolve_catalog_plan` / `resolve_data_plan` split from Question 4 to §6's file table.

R7 Fix two inventory errors and one omission

§1 names the telemetry class `ExecutionTelemetry`; the class is `CausalTelemetry`. §1 also omits `utils/runner/checkpoint.py` (`DailyStateCheckpointer`), which Fixture 1 requires for both its config identity and its `--resume` path. Add checkpoint/resume to §5 as an explicitly unsupported case if B1 will not carry it.

WHAT B0 GETS RIGHT

Worth preserving through the revision

- The core judgement — reuse `backtests/nt_runtime/` and `utils/runner/` rather than creating a `common/` package — is correct. Both legacy scripts duplicate `create_instrument()` and the venue block nearly verbatim; that duplication is real and worth removing.
- Mirroring `modes/collect.py` as `modes/backtest.py` keeps the new path parallel to a structure that has now survived six red-team rounds.
- The §5 exclusions (non-futures, sweeps, tick/MBP-1, live) are accurate and honestly scoped.
- Both legacy scripts already add 1s before 1m, so `add_bars_causal_order` is a faithful consolidation rather than a behavioral change.
- The venue settings identified for the `engine_builder` change (`bar_execution`, `bar_adaptive_high_low_ordering`) are exactly the two that differ from current `build_engine`.

Red-team review of `BACKTEST_HARNESS_B0_BOUNDARY.md` · 2026-08-15 · verdict **BLOCKED**, 7 required revisions, all document-level. Verified against `run_staged_backtest.py`, `run_w4_backtest.py`, `score_fanning_strategy.py`,

`w4_exit_strategy.py`, `baseline_flip_parity/strategy.py`, the Gemini study contract and the current 53-file execution closure. No code was modified.